Multi-Task Learning in PRoNTo

Strategy

In PRoNTo, models are specified independently as Single-Task Learning (STL) problems then can be combined at the 'run model stage'. During this step, a new model is created (the MTL model) that includes the chosen MTL machine and refers to all included STL models.

The idea behind this reasoning is that:

1. Users will want to compare STL with MTL, so STL model needs to be specified (and maybe run) anyways.
2. It gives a lot of freedom on which operations or feature sets to choose from in each model.

STL models can have different ID matrices, targets or operations when being combined.

Limitations

- All selected models must include the exact same features
- STL models have to be all regression or all classification problems
- All selected models must have the same number of folds
  - ➔ Careful! There is no check ensuring that data selected as train in fold 1 of model 1 has not been selected as test in fold 1 of model 2. This would create a bias in the model.

Non-Kernel implementation

MALSAR

The MALSAR toolbox (https://github.com/jiayuzhou/MALSAR, by Jiayu Zhou, Jianhui Chen and Jieping Ye, *Zhou, Jiayu, Jianhui Chen, and Jieping Ye. "MALSAR: Multi-task learning via structural regularization." Arizona State University (2011). http://www.MALSAR.org.*) was interfaced in PRoNTo. It includes quite a few algorithms for both regression and classification. The formulations interfaced in PRoNTo are:

Classification:

1. Lasso (Logistic_Lasso), arguments: rho1, the common sparsity across tasks and (optional) rho2 for elastic-net
2. L2,1- norm (Logistic_L21), arguments: arguments: rho1, the sparsity among tasks and (optional) rho2 for elastic-net
3. Trace-norm regularization, (Logistic_Trace), arguments: lambda for

Regression:

4. Lasso (Least_Lasso), arguments: rho1, the common sparsity across tasks and (optional) rho2 for elastic-net
5. L2,1- norm (Least_L21), arguments: rho1 and (optional) rho2
6. Trace-norm regularization, (Least_Trace), arguments: lambda to control the rank minimization problem.
7. Dirty model (Least_Dirty), arguments: rho1 for regularization of group sparse component, rho2 for regularization on task-specific sparse component. (both mandatory)

Calling the machine:

Specify the valued arguments in a vector ([rho1, rho2]) and a string argument as '-s index_machine -args ', index_machine being the number of the formulation to use.

For example: '-s 2 -args ' refers to the L2,1-norm binary classification while '-s 7 –args ' refers to the Dirty model. Please note the white spaces around the number and after the '-args ', where the valued arguments will be appended before being passed to the machine.

It would of course be possible to extend these calls in the prt_machine_MTL_MALSAR to interface more formulations from this toolbox. They have potentially interesting longitudinal models or multi-stage MTL.

Kernel MTL

PRoNTo interfaces the implementation from Ciliberto et al., 2015 that can be found at https://github.com/cciliber/matMTL/issues and is based on their publication Ciliberto, Carlo, Tomaso Poggio, and Lorenzo Rosasco. Convex Learning of Multiple Tasks and their Structure. International Conference on Machine Learning (ICML), 2015 (arXiv here: https://arxiv.org/pdf/1504.03101.pdf). It is a regression kernel formulation, with a couple of options:

1. 'trace': implements the L2,1-norm formulation in Argyriou et al., 2008. It has one hyper-parameter.
2. 'ind': independent models with no task relatedness, no hyper.
3. 'fix': implements the variance formulation of MTL that forces all task parameters to stay close to a common mean (Evgeniou and Pontil, 2004). How far away from the mean they can stray is controlled by a second hyper-parameter, gamma, that can be optimized (a default value of 0.1 is provided).

The code includes a primal and a dual formulation but only the dual is used here. To access the machine, use the function prt_machine_KernelMTL.m and specify string arguments as '-s index_machine –args '.

MTL with cross-validation in PRoNTo

To run an MTL model in PRoNTo, you first need to specify the models for the different tasks independently. Ensure that all models have the same number of folds. The next steps need to be performed in the workspace:

```
load('PRT.mat')
% Specify new MTL model
%------------------------------
% Requires user input:
iSTL = index of STL models; %vector with indices of models to combine
PRT.model(im+1).model_name = 'Name_of_your_MTLmodel';
in.fname = 'PRT.mat'; % or full path to PRT if not in the working folder

% Then copy from this:
im = numel(PRT.model);
PRT.model(im+1).models_MTL = iSTL;
PRT.model(im+1).input.machine.function = 'prt_machine_KernelMTL'; %Kernel version
PRT.model(im+1).input.machine.s_args = '-s 1 –args '; % Using L2,1-norm
PRT.model(im+1).input.machine.args = 1; %In case no tuning is performed
PRT.model(im+1).input.use_kernel = 0; % This seems counterintuitive but we need to get all the data from all tasks to compute the overall kernel for this machine. This is intuitive for the MALSAR machine.
```

```
PRT.model(im+1).input.type = 'regression'; % or classification
PRT.model(im+1).input.fs = PRT.model(iSTL(1)).input.fs; %as all STL models must have the same features
PRT.model(im+1).input.use_nested_cv = 1; % To tune, otherwise 0
PRT.model(im+1).input.nested_param = 10.^[-2:3]; % Hyper-parameter range, for kernel MTL with variance, use a cell {10.^[-2:3],0.01:0.01:0.1} for example.
PRT.model(im+1).input.cv_type_nested = 'loso'; % or 'losgo', or 'lobo' or 'locbo' or 'loro'
PRT.model(im+1).input.cv_k_nested = 2; %any integer, 0 default for LOO
save('PRT.mat','PRT');
% Run
% -----------------------------------
in.model_name = PRT.model(im+1).model_name;
in.models_MTL = {PRT.model(iSTL).model_name}; % The names, in a cell array, of models to combine
out = prt_cv_MTL(PRT,in); % runs MTL
```

Note: for the moment, the nested CV is derived from the MTL model and applied to all tasks but this could be extended to include the nested CV from each independent model. This would include cases with e.g. more than one image per subject in a task, and single images per subject in another.

Note 2: Copy-pasting into Matlab doesn't work as the characters for '' or – are not the same.

Outputs

The outputs of the model are saved in the same fields as for the STL, but predictions and targets are now cells of size number of tasks and the weights are stored in a matrix of size number of features x number of tasks (output.fold.w). For the kernel MTL, the dual coefficients are saved in a matrix of size number of total training samples x number of tasks for the kernel version (output.fold.coeffs). In the primal, the bias is saved as well in output.fold.b. Additional outputs are saved in the 'output.fold.others' field, including the P and Q weights for the Dirty Model, or the A matrix imposing task-relatedness in the kernel formulation.

During prediction, the sample is passed onto its specific task and the prediction is made based on this task's weights only. In the future, for samples pertaining to new or untrained tasks, one could however imagine to make a prediction for each task but average or vote the final prediction across all tasks.

Stats are computed within and across folds, as well as within and across tasks (see prt_stats.m). Please note that these averages do not weight the tasks on their numbers of samples (this could be improved).

Weights

Just run the compute weights (batch or GUI) to obtain the images of the weights, one image per task. The name of the images are 'weights_*name of MTL model_name of STL task*' (+ name of class if multiclass but no machine dealing with this as of now).

Weights can be displayed in the main window.

Permutations

Permutations need to be first run for each STL model separately, and saved (i.e. ticking the 'save permutation parameters' option).

A code can then be called from the workspace to perform permutations:

```
Out = prt_permute_MTL(PRT,mid,fname,maxnp)
```

The inputs are:

PRT: loaded PRT structure

mid: index of MTL model to compute permutations for

fname: full path and name to PRT.mat file (for saving the output)

maxnp: maximum number of permutations to run

The number of permutations run will be the minimum of maxnp and the number of permutations run on the STL models.

Important note: the code uses prt_cv_MTL (unlike the prt_permutations code that re-codes this). This however means that the permuted targets temporarily replace the true targets for the STL models in the PRT.mat. This means that if, for some reason, the code is interrupted, the PRT.mat will be corrupted (more precisely, the targets of the STL models).

An easy fix is:

```
for i=1:length(iSTL)
        sidx = PRT.model(im).input.samp_idx;
        PRT.model(im).input.targets =PRT.model(im).input.targ_allscans(sidx);
end
save('PRT.mat','PRT') % being careful with path
```